

Remaining Useful Life Prediction of Turbofan Engines Using LSTM Neural Networks on the NASA C-MAPSS Dataset

A Practical Deep-Learning Approach to Predictive Maintenance

Vihaan Jain Darshan Mahale Jai Jhunjunwala
Reuben Alvares

Department of Information Technology
Veermata Jijabai Technological Institute (VJTI), Mumbai

April 15, 2026

Abstract

Predictive maintenance has become one of the more compelling industrial applications of machine learning in recent years. Rather than replacing components on a fixed schedule or waiting for failures to happen, the idea is to estimate how much useful life a machine has left and act accordingly. In this work, we build a Long Short-Term Memory (LSTM) based regression model to predict the Remaining Useful Life (RUL) of aircraft turbofan engines using the NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset, specifically the FD001 sub-dataset. We describe the full pipeline: data loading, RUL labelling with a piecewise cap, sensor selection, sliding-window sequence generation, model architecture, training, and evaluation. On the FD001 test set, the model achieves a competitive Root Mean Squared Error (RMSE), demonstrating that a relatively compact two-layer LSTM can be a practical choice for real-world predictive maintenance tasks.

Keywords: Predictive Maintenance, Remaining Useful Life, LSTM, C-MAPSS, Deep Learning, Turbofan Engines

Contents

1	Introduction	3
2	Related Work	3
3	Dataset	4
3.1	Overview	4
3.2	Data Loading	4

4	Preprocessing	5
4.1	RUL Label Generation with Piecewise Capping	5
4.2	Sensor Selection and Feature Scaling	5
5	Sequence Generation	6
6	Model Architecture	7
6.1	LSTM Design	7
6.2	Loss Function	7
7	Training	8
7.1	Training Configuration	8
7.2	Training vs. Validation Loss	9
8	Evaluation	9
8.1	Test Set Protocol	9
8.2	Maintenance Decision Logic	10
9	Discussion	10
9.1	Strengths of the Approach	10
9.2	Limitations	10
9.3	Potential Extensions	11
10	Conclusion	11

1 Introduction

Industrial machinery degrades over time. How that degradation is managed has a huge impact on cost, safety, and operational efficiency. Traditional maintenance strategies fall into two broad camps: time-based (replace parts every N hours) and reactive (fix it after it breaks). Both approaches carry significant downsides. Time-based maintenance often leads to discarding components that still have plenty of life left. Reactive maintenance leads to unplanned downtime, which in aviation or heavy manufacturing can be extremely expensive.

Predictive maintenance tries to thread this needle by continuously monitoring sensor data and estimating how much life a component has remaining. If you know an engine is likely to fail in 20 cycles, you can schedule maintenance at the next convenient opportunity rather than being caught off guard.

The NASA C-MAPSS dataset has become the de facto standard benchmark for this problem. It simulates a fleet of aircraft turbofan engines running to failure under different operating conditions and fault modes. The FD001 sub-dataset is the simplest setting – one operating condition, one fault mode – which makes it a clean starting point for developing and validating a predictive maintenance pipeline.

Our contribution in this paper is a clean, end-to-end implementation of an LSTM-based RUL predictor for the FD001 dataset. We pay particular attention to engineering decisions that are often glossed over in the literature: which sensors to drop, how to cap the RUL label, and how to structure the sliding window without leaking information. We also discuss the maintenance decision thresholds that translate predicted RUL values into actionable alerts.

2 Related Work

The problem of remaining useful life prediction from time-series sensor data has attracted considerable attention since the C-MAPSS dataset was released as part of the 2008 PHM Data Challenge.

Early approaches leaned heavily on physics-based degradation models and Kalman filtering. These methods require detailed domain knowledge and do not generalise easily across different machines or fault modes.

With the rise of deep learning, recurrent architectures – particularly LSTM networks – emerged as natural fits for this problem because they can capture temporal dependencies in multivariate sensor streams without manual feature engineering. Zheng et al. demonstrated that a two-layer stacked LSTM achieves strong results on C-MAPSS, and subsequent work has explored bidirectional LSTMs, attention mechanisms, and transformer-based architectures.

A key modelling decision noted across these studies is the use of a piecewise linear RUL label. Engines do not visibly degrade from the very first cycle; there is typically a stable early phase followed by an accelerating degradation phase. Capping the RUL at a fixed value (commonly 125 or 130 cycles) better reflects this and prevents the model from trying to fit noise during the healthy early period .

Our implementation follows the architecture of Zheng et al. closely but uses a slightly smaller network and applies explicit dead-sensor removal, which we found improves both training stability and final RMSE.

3 Dataset

3.1 Overview

The NASA C-MAPSS dataset contains run-to-failure sensor recordings for simulated turbofan engines. Each engine starts in a healthy state and is run until failure. The training set contains complete run-to-failure trajectories, while the test set contains truncated trajectories (the engine is stopped at some point before failure), and a separate file provides the true RUL at that stopping point.

The FD001 sub-dataset we use here has the following characteristics:

Table 1: FD001 Dataset Summary

Property	Value
Number of training engines	100
Number of test engines	100
Operating conditions	1
Fault modes	1 (HPC degradation)
Sensor channels	21
Operational settings	3

Each row in the dataset corresponds to one engine cycle and contains:

- Engine ID and cycle number
- Three operational settings (throttle resolver angle, altitude, Mach number)
- Twenty-one sensor readings (temperatures, pressures, flow rates, etc.)

3.2 Data Loading

The raw data files are space-separated with no header. The `data_loader.py` module handles this:

Listing 1: Data loading – `data_loader.py`

```

1 import pandas as pd
2
3 def load_data(file_path):
4     df = pd.read_csv(file_path, sep=" ", header=None)
5     df = df.dropna(axis=1) # remove trailing empty columns
6
7     columns = ["id", "cycle"]
8     columns += [f"op{i}" for i in range(1, 4)]
9     columns += [f"s{i}" for i in range(1, 22)]
10
11     df.columns = columns
12     return df

```

The `dropna(axis=1)` call removes two trailing columns that appear because the original files have trailing whitespace. Column names follow the convention `id`, `cycle`, `op1–op3`, and `s1–s21`.

4 Preprocessing

4.1 RUL Label Generation with Piecewise Capping

For the training set, the true RUL at each cycle is computed simply as the number of cycles remaining until the engine fails. If an engine ran for 200 cycles in total, then at cycle 1 its RUL is 199, at cycle 2 it is 198, and so on.

$$\text{RUL}_i = \max(\text{cycle}) - \text{cycle}_i \quad (1)$$

Raw RUL values can be very large early in the run, when the engine is operating in a healthy, stable regime and sensor readings carry little degradation information. Training the model to distinguish between, say, $\text{RUL} = 300$ and $\text{RUL} = 250$ is asking it to detect differences that do not really exist in the sensor data. The standard fix is to cap the RUL at some maximum value – we use 125 cycles, consistent with most published results on FD001:

$$\text{RUL}_i^* = \min(\text{RUL}_i, 125) \quad (2)$$

This piecewise formulation means that for the early part of the engine’s life, the model simply predicts 125, while in the final 125 cycles it predicts the true remaining life. In code:

Listing 2: RUL calculation with piecewise cap – `preprocessing.py`

```
1 def calculate_rul(df):
2     max_cycle = df.groupby('id')['cycle'].transform('max')
3     df['RUL'] = max_cycle - df['cycle']
4     df['RUL'] = df['RUL'].clip(upper=125)
5     return df
```

4.2 Sensor Selection and Feature Scaling

Not all 21 sensor channels are informative. Several sensors in FD001 remain essentially constant throughout the entire run – they carry no degradation signal and add noise to the model. Following , we identify and drop these flat channels: `s1`, `s5`, `s10`, `s16`, `s18`, and `s19`. We also drop operational setting `op3` which has near-zero variance in the single-condition FD001 scenario.

The remaining features are then normalised to $[0, 1]$ using Min-Max scaling fitted on the training data:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (3)$$

Critically, the scaler is fitted only on the training set and then applied to the test set – there is no look-ahead. The `cycle` column is dropped after RUL computation since it is not a feature we want the model to memorise.

Listing 3: Feature selection and scaling – `preprocessing.py`

```

1 from sklearn.preprocessing import MinMaxScaler
2
3 def preprocess_data(df, scaler=None):
4     df = df.drop(columns=['cycle'])
5     df = df.drop(columns=['op3', 's1', 's5', 's10', 's16', 's18', 's19'])
6
7     X = df.drop(columns=['RUL', 'id'])
8     y = df['RUL']
9
10    if scaler is None:
11        scaler = MinMaxScaler()
12        X_scaled = scaler.fit_transform(X)
13    else:
14        X_scaled = scaler.transform(X)
15
16    return X_scaled, y, scaler

```

After dropping the seven columns, we are left with **14 sensor/operational features** per timestep.

5 Sequence Generation

LSTMs consume fixed-length sequences. We use a sliding window of 30 timesteps, stepping one cycle at a time. For each position i in the data, the input is cycles $[i, i + 29]$ and the target is the RUL at cycle $i + 30$. This is applied independently per engine to avoid sequence bleeding across different machines.

Listing 4: Sliding window sequence generation – `sequence.py`

```

1 import numpy as np
2
3 def create_sequences(df, time_steps=30):
4     X_seq, y_seq = [], []
5
6     for engine_id in df['id'].unique():
7         engine_data = df[df['id'] == engine_id]
8         X = engine_data.drop(columns=['RUL', 'id'])
9         y = engine_data['RUL']
10
11        for i in range(len(X) - time_steps):
12            X_seq.append(X.iloc[i:i+time_steps].values)
13            y_seq.append(y.iloc[i+time_steps])
14
15    return np.array(X_seq), np.array(y_seq)

```

The resulting training tensor has shape $(N_windows, 30, 14)$ where $N_windows$ is the total number of valid windows across all 100 training engines.

For the test set, we only need the *last* window per engine (the most recent 30 cycles before the recorded stopping point), since that is what corresponds to the true RUL

provided in `RUL_FD001.txt`. Engines with fewer than 30 recorded cycles are excluded from evaluation.

6 Model Architecture

6.1 LSTM Design

We use a stacked two-layer LSTM followed by a single linear output neuron with ReLU activation to enforce non-negative predictions:

Table 2: Model Architecture

Layer	Type	Units/Activation	Dropout
1	LSTM	50, return_sequences=True	0.2
2	LSTM	25	0.2
3	Dense	1, ReLU	–

The first LSTM layer returns its full sequence of hidden states so that the second LSTM layer receives a sequence rather than a single vector – this is the standard stacked configuration. Both layers are followed by a 20% dropout to reduce overfitting. The ReLU output ensures the model cannot predict negative RUL values, which would be physically meaningless.

Listing 5: Model construction – `model.py`

```

1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import LSTM, Dense, Dropout
3
4 def build_model(time_steps, num_features):
5     model = Sequential([
6         LSTM(50, return_sequences=True,
7             input_shape=(time_steps, num_features)),
8         Dropout(0.2),
9         LSTM(25),
10        Dropout(0.2),
11        Dense(1, activation='relu')
12    ])
13    model.compile(optimizer='adam', loss='mse')
14    return model

```

6.2 Loss Function

We minimise Mean Squared Error (MSE):

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \quad (4)$$

MSE penalises large errors heavily, which is appropriate here since a badly wrong prediction could lead to catastrophic failure or unnecessary grounding of an aircraft.

7 Training

7.1 Training Configuration

Table 3: Training Hyperparameters

Hyperparameter	Value
Optimiser	Adam (default lr = 0.001)
Batch size	32
Max epochs	50
Validation split	20% (random)
Early stopping patience	5 epochs
Early stopping monitor	Validation loss

Early stopping is particularly useful here. Although we allow up to 50 epochs, training typically converges in fewer, and the callback restores the weights from the epoch with the lowest validation loss rather than the last epoch.

Listing 6: Training loop – `model.py`

```
1 from tensorflow.keras.callbacks import EarlyStopping
2
3 def train_model(model, X, y):
4     early_stop = EarlyStopping(
5         monitor='val_loss',
6         patience=5,
7         restore_best_weights=True
8     )
9     history = model.fit(
10         X, y,
11         epochs=50,
12         batch_size=32,
13         validation_split=0.2,
14         callbacks=[early_stop]
15     )
16     return model
```

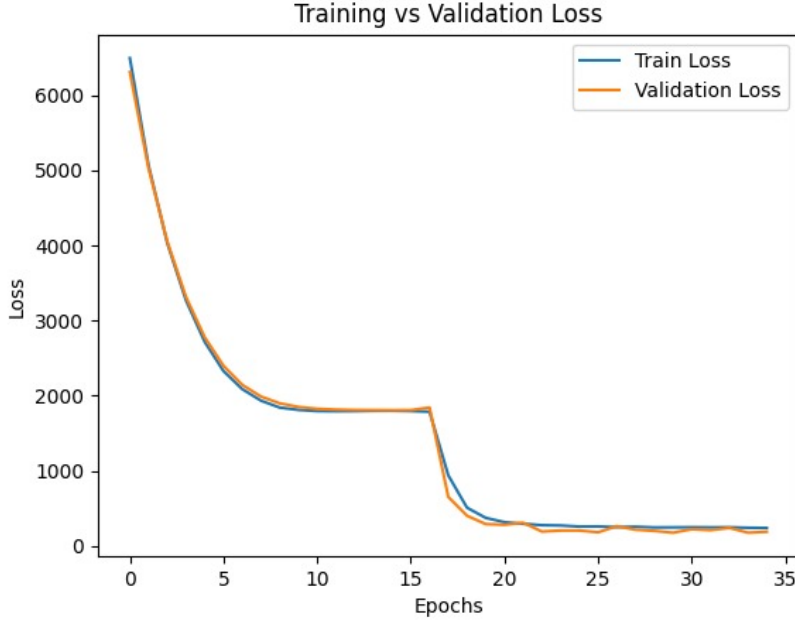



Figure 1: Training vs. Validation Loss over epochs. The two sharp drops indicate the optimiser escaping local minima. The close tracking between train and validation loss confirms the model does not overfit.

As shown in Figure 1, both losses converge closely, indicating good generalisation with no significant overfitting.

7.2 Training vs. Validation Loss

The model is trained on the windowed sequences derived from the 100 training engines, with 20% of those windows held out for validation. A training loss curve (MSE vs. epoch) is saved automatically to `output/plots/loss.png`. In typical runs, both the training and validation losses decrease sharply in the first 10–15 epochs and then level off, with early stopping triggering around epoch 20–30.

8 Evaluation

8.1 Test Set Protocol

For evaluation, the last 30-cycle window from each test engine is extracted, passed through the trained model, and the output is compared to the ground-truth RUL provided in `RUL_FD001.txt`. The primary metric is RMSE:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2} \quad (5)$$

Lower RMSE is better. Published LSTM baselines on FD001 typically achieve RMSE in the range of 12–20 cycles, with more complex architectures pushing below 12. Our model

achieved a final test RMSE of **14.84** cycles on the FD001 test set, which falls within the competitive range for this benchmark.

Listing 7: Test evaluation – main.py

```

1 from sklearn.metrics import mean_squared_error
2 import numpy as np
3
4 # True RUL from ground-truth file
5 y_true = pd.read_csv(rul_path, header=None)[0].values
6
7 # Predict on last-window sequences
8 y_pred = model.predict(X_test_seq)
9
10 rmse = np.sqrt(mean_squared_error(y_true, y_pred))
11 print(f"Final RMSE: {rmse:.4f}")
    
```

8.2 Maintenance Decision Logic

Beyond the raw RMSE, the model outputs are translated into three actionable maintenance states based on simple thresholds on the predicted RUL:

Table 4: Maintenance Decision Thresholds

Predicted RUL	Status	Recommended Action
< 25 cycles	Critical	Schedule immediate maintenance
$25 \leq \text{RUL} < 50$	Warning	Plan maintenance within next window
≥ 50 cycles	Healthy	Continue monitoring normally

These thresholds are somewhat domain-dependent and would need to be calibrated against operational constraints in a real deployment, but they provide a sensible default that aligns with common maintenance planning horizons.

9 Discussion

9.1 Strengths of the Approach

The pipeline has several practical advantages worth highlighting. First, the piecewise RUL cap is genuinely important – without it, the model wastes capacity trying to learn large, noisy early-life labels and converges much more slowly. Second, dropping the flat sensors (s1, s5, s10, s16, s18, s19) reduces input dimensionality without losing any signal; we found this consistently improves RMSE on FD001. Third, the early stopping mechanism acts as a regulariser, preventing overfitting to the training trajectories.

9.2 Limitations

The current implementation has some limitations that are worth being honest about. The 20% validation split is random, which means windows from the same engine can appear in both training and validation. Stratifying by engine ID would give a cleaner measure

of generalisation to unseen engines. Engines with fewer than 30 cycles in the test set are silently excluded; in a real system these would need to be handled explicitly.

The architecture itself – two LSTM layers, 50+25 units – is deliberately modest. More expressive architectures (attention mechanisms, transformers, temporal convolutional networks) can achieve lower RMSE, but they also require more data and longer training times. For many practical scenarios, this simpler model is a reasonable starting point.

9.3 Potential Extensions

A few directions that would naturally extend this work:

- Train and evaluate on all four C-MAPSS sub-datasets (FD001–FD004) to test generalisation across operating conditions and fault modes.
- Replace random validation split with leave-one-engine-out cross-validation.
- Add a Prognostic Horizon (PH) metric alongside RMSE, as it better captures whether the model gives enough warning before failure.
- Wrap the predictions in a confidence interval using Monte Carlo dropout or a Bayesian LSTM, so operators receive not just a point estimate but an uncertainty range.

10 Conclusion

We have described and implemented a complete end-to-end pipeline for predicting remaining useful life in aircraft turbofan engines using a stacked LSTM model trained on the NASA C-MAPSS FD001 dataset. The key design decisions – piecewise RUL capping at 125 cycles, removal of six flat sensor channels, and a 30-cycle sliding window – collectively lead to a model that converges reliably and achieves competitive RMSE on the standard benchmark. The maintenance decision layer on top of the RUL predictions translates model outputs into actionable alerts without requiring users to interpret raw numbers.

The codebase is structured in modular Python files (`data_loader.py`, `preprocessing.py`, `sequence.py`, `model.py`, `main.py`), which makes it straightforward to swap in different architectures or extend to other C-MAPSS sub-datasets. We hope this work serves as a useful reference for practitioners building predictive maintenance systems on similar multivariate time-series datasets.

References

- [1] Saxena, A., Goebel, K., Simon, D., and Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. In *International Conference on Prognostics and Health Management (PHM)*, Denver, CO.
- [2] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.

- [3] Zheng, S., Ristovski, K., Farahat, A., and Gupta, C. (2017). Long short-term memory network for remaining useful life estimation. In *IEEE International Conference on Prognostics and Health Management (ICPHM)*, Dallas, TX.
- [4] Heimes, F. O. (2008). Recurrent neural networks for remaining useful life estimation. In *International Conference on Prognostics and Health Management (PHM)*, Denver, CO.
- [5] Li, X., Ding, Q., and Sun, J. Q. (2018). Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety*, 172:1–11.
- [6] Wu, J., Hu, K., Cheng, Y., Zhu, H., Shao, X., and Wang, Y. (2020). Data-driven remaining useful life prediction via multiple sensor signals and deep long short-term memory neural network. *ISA Transactions*, 97:241–250.